Data 2 - Trees

Design Programming 2

Python / General Web data (Javascript, JSON, etc)

Dictionaries

```
{ // A dictionary or object
  "Key 1": "Object 1",
  "Key 2": "Value 1"
}
```

Arrays

```
[   # An Array
 “Item 1",
 “Item 2",
 3,
 Object_1
]

Or

[ “Item 1", “Item 2", 3, Object_1]
```

Python / General Web data (Javascript, JSON, etc)

Dictionaries & arrays

```
[   // An Array
  {
    "Key 1": "Object 1",
    "Key 2": "Value 1"
  },
  {
    "Key 1": "Object 2",
  },
  {
    "Key 2": "Value 3",
    "Key 1": "Object 3",
    "Key 3": "Value 4" // Flexible
  }
]
```

Python / General Web data (Javascript, JSON, etc)

Nested objects

```
[   // An Array
  {
    "Key 1": “Value 1",
    "Key 2": "Value 2"
  },
  {
    "Key 2": "Value 3",
    "Key 1": “Value 4",
    "Key 3": {

                  "Key 3a": “Value 1",
                  "Key 3b": “Value 2",

               }

  }
]
```

Accessing arrays (see w3c Python Arrays)

Get the value of the first array item:

```
x = cars[0]
```

```
cars[0] = "Toyota"
```

Return the number of elements in the `cars` array:

```
x = len(cars)
```

Print each item in the `cars` array:

```
for x in cars:
  print(x)
```

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Dictionaries (see W3c Python Dictionaries)

Creating

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Creating – using dict()

```
thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

Accessing individual items

```
thisdict["brand"]
```

```
thisdict.get("model")
```

```
thisdict.keys()
```

Accessing all items

```
thisdict.values()
```

```
thisdict = {
    "brand":"Ford",
    "model":"Mustang",
    "year":1964
    }

keys = thisdict.keys()
values = thisdict.values()

for i in keys:
    print(thisdict[i])


for x, y in thisdict.items():
  print(x, y)
```

Dictionaries (see W3c Python Dictionaries)

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

“Structured” vs “Unstructured” Data

Sometimes data associated with instances is consistent across similar instances. This sort of data is (somewhat erroneously) called “structured data” or “tabular data”, and we can represent it in a table, where
- Rows are individual data instances
- columns are the different (but consistent) key – value combinations:

Sometimes a specific parameter is identified as the ID of the instance. In that case this value must be unique for each instance:

ID	brand	model	year
00001	Ford	Mustang	1964
00002	Ford	Mustang	1967
00003	BMW	I5	2022
00004	BMW	I3	2024
00005	Volkswagen	Rabbit	1990

This is equivalent to - but not the same as - the following

```
[
   {
         ID :  00001,
         brand: “Ford”,
         model: “Mustang”,
         year: 1964
   },
   {
         ID :  00002,
         brand: “Ford”,
         model: “Mustang”,
         year: 1967
   },
. . .
```

Tabular data - serialization

Tabular data can be “serialized” as text with defined character separators between fields.

Serialize (roughly) write data out to a string or stream

ID	brand	model	year
00001	Ford	Mustang	1964
00003	BMW	I5	2022
00005	Volkswagen	Rabbit	1990

Could be serialized as follows:

ID,brand,model,year <cr>
00001,Ford,Mustang,1964 <cr>
00003,BMW,I5,2022 <cr>
00005,Volkswagen,Rabbit,1990 <cr>

This serialization has commas between fields and is known as CSV (comma separated value) format.

Data serialization: to JSON

json.loads("aString")

json.dumps(anObject)

```
import json

# some JSON:
# JSON TEXT
xText =  '{ "name":"John", "age":30, "city":"New York"}'
print (type(xText)) # <class 'str'>

yData = {
    "name": "Sally",
    "age" : 25,
    "city": "Washington, DC"
}
print (type(yData))  # <class 'dict'>

# parse (load) TEXT to DATA
xData = json.loads(xText)
# write (dump) DATA to TEXT
yText = json.dumps(yData)

# the result is a Python dictionary:
print(xData["name"])
print(yData["name"])
```

Data vs. Objects

Data is naturally organized in name / value or key / value pairs:

Key	Value
var1	“My text”
var2	“My other value”

A collection of name / value pairs is sometimes called a **dictionary**. It’s a very powerful and general concept so you will see either “analogies” of this concept in other contexts, or in fact actual reframing of the concept in other contexts.

For example, an **object instance** shares a lot with this dictionary concept, and objects and dictionaries are sometimes (not always) used interchangeably.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
```

```
class Car:
  def __init__(brand, model, year):
    self.brand = brand
    self.model = model
    self.year = year

c1 = Car(“Ford”, “Mustang”, 1964)
```

c1.brand = “Ford”
c1.model = “Mustang”
c1.year = 1964

Dictionaries (see W3c Python Dictionaries)

Creating

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Creating – using dict()

```
thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

Accessing individual items

```
thisdict["brand"]
```

```
thisdict.get("model")
```

Accessing all items

```
thisdict.keys()
```

```
thisdict.values()
```

```
thisdict = {
    "brand":"Ford",
    "model":"Mustang",
    "year":1964
    }

keys = thisdict.keys()
values = thisdict.values()

for i in keys:
    print(thisdict[i])
```

```
for x, y in thisdict.items():
  print(x, y)
```

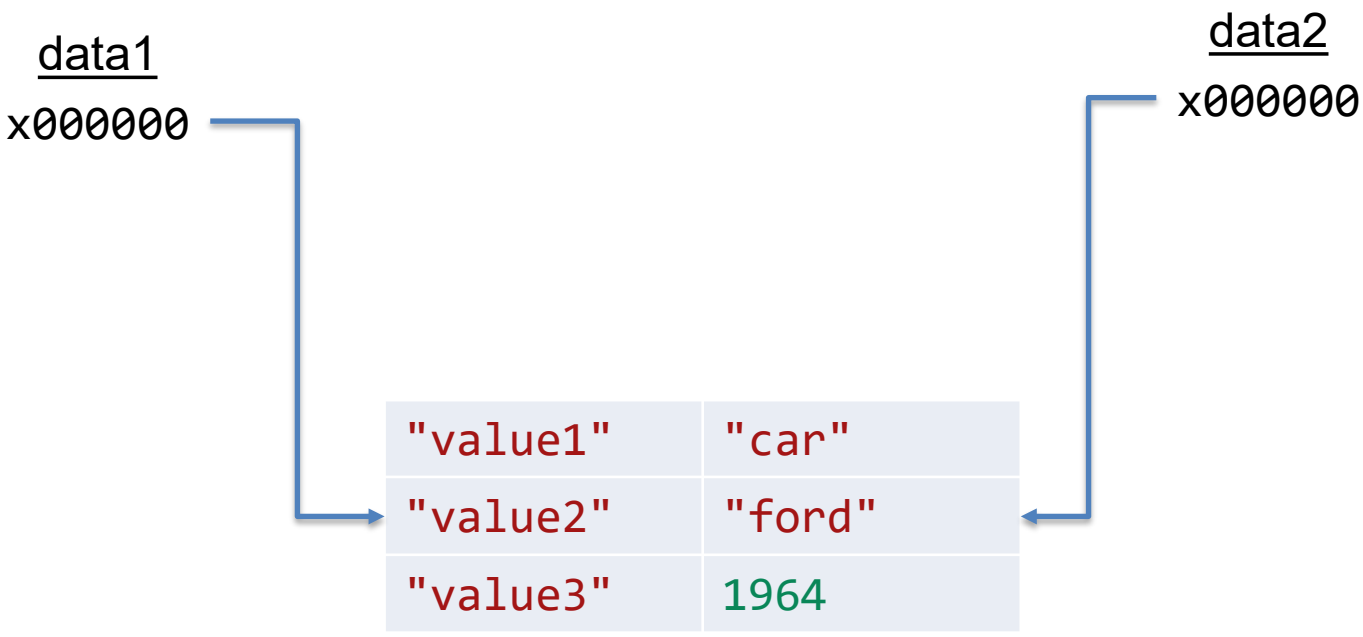
Copy vs. Deep copy

data1

```
{
    "value1": "car",
    "value2": "ford",
    "value3": 1964,
}
```

data2

```
{
    "value1": "car",
    "value2": "ford",
    "value3": 1964,
}
```



Dictionaries (see W3c Python Dictionaries)

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Object Data

Similarly, data can be saved **on specific objects** using SetUserText and related functions,

and can be retrieved using GetUserText.

This data is persistent on the object, so it can be retrieved after the file is saved and re-opened later.

```
# setUserText.py

import rhinoscriptsyntax as rs
ref = rs.GetObject("select the object")

rs.SetUserText(ref, "MyEntry1", "1")
rs.SetUserText(ref, "MyEntry2", "car")
rs.SetUserText(ref, "MyEntry3", "purple")
```

```
import rhinoscriptsyntax as rs
ref = rs.GetObject("select the object")

value = rs.GetUserText(ref, "MyEntry1")
print(value)
value = rs.GetUserText(ref, "MyEntry2")
print(value)
value = rs.GetUserText(ref, "MyEntry3")
print(value)
```

Document Data

Data can be saved to the document using SetDocumentData and related functions,

and can be retrieved using GetDocumentData.

This data is persistent on the file, so it can be retrieved after the file is saved and re-opened later.

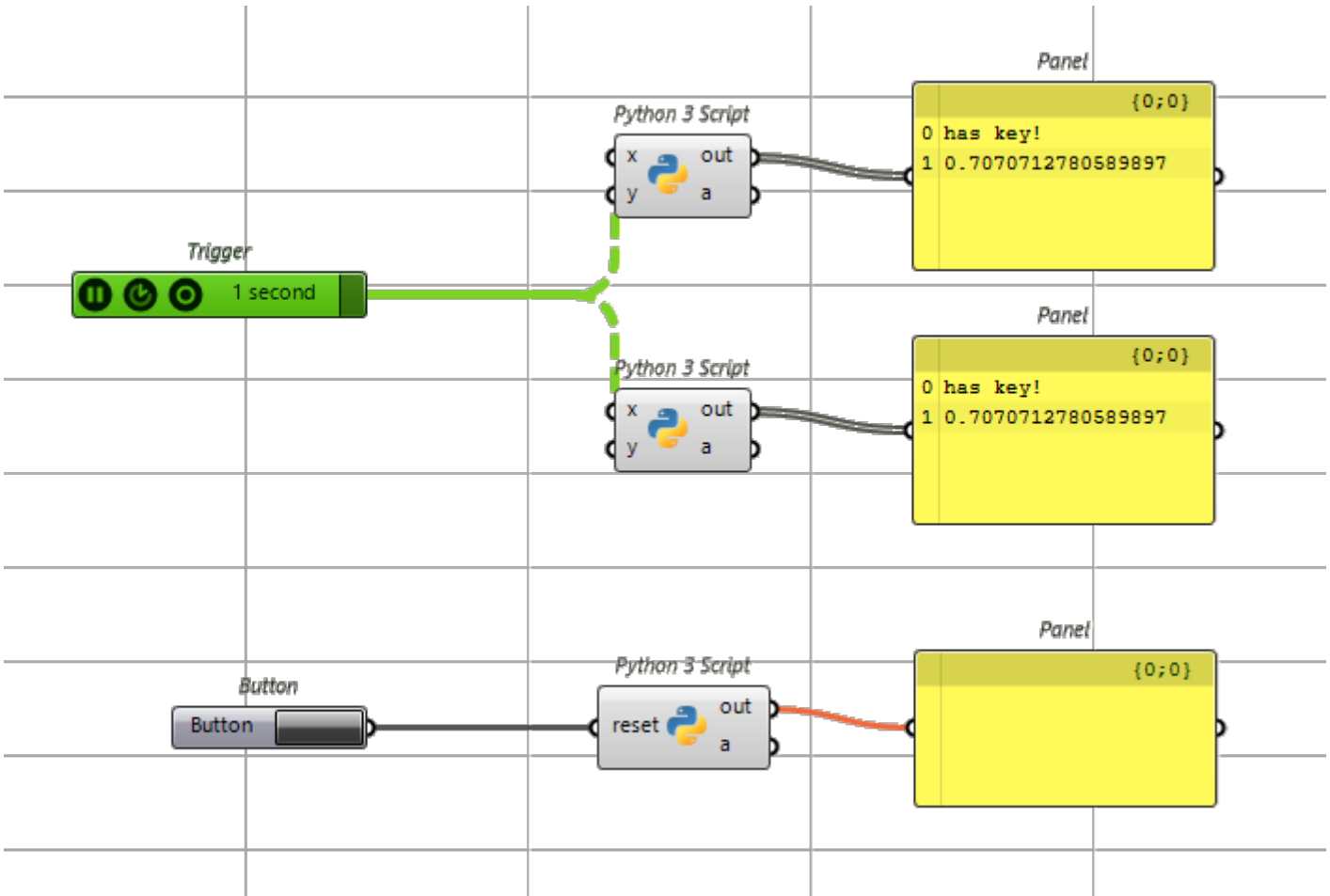
```
# setDocumentData.py

import rhinoscriptsyntax as rs
rs.SetDocumentData("DRS", "MyEntry1", "1")
rs.SetDocumentData("DRS", "MyEntry2", "3")
rs.SetDocumentData("DRS", "MyEntry3", "7")
```

```
# getDocumentData.py

import rhinoscriptsyntax as rs
value = rs.GetDocumentData("DRS", "MyEntry1")
print(value)
value = rs.GetDocumentData("DRS", "MyEntry2")
print(value)
value = rs.GetDocumentData("DRS", "MyEntry3")
print(value)
```

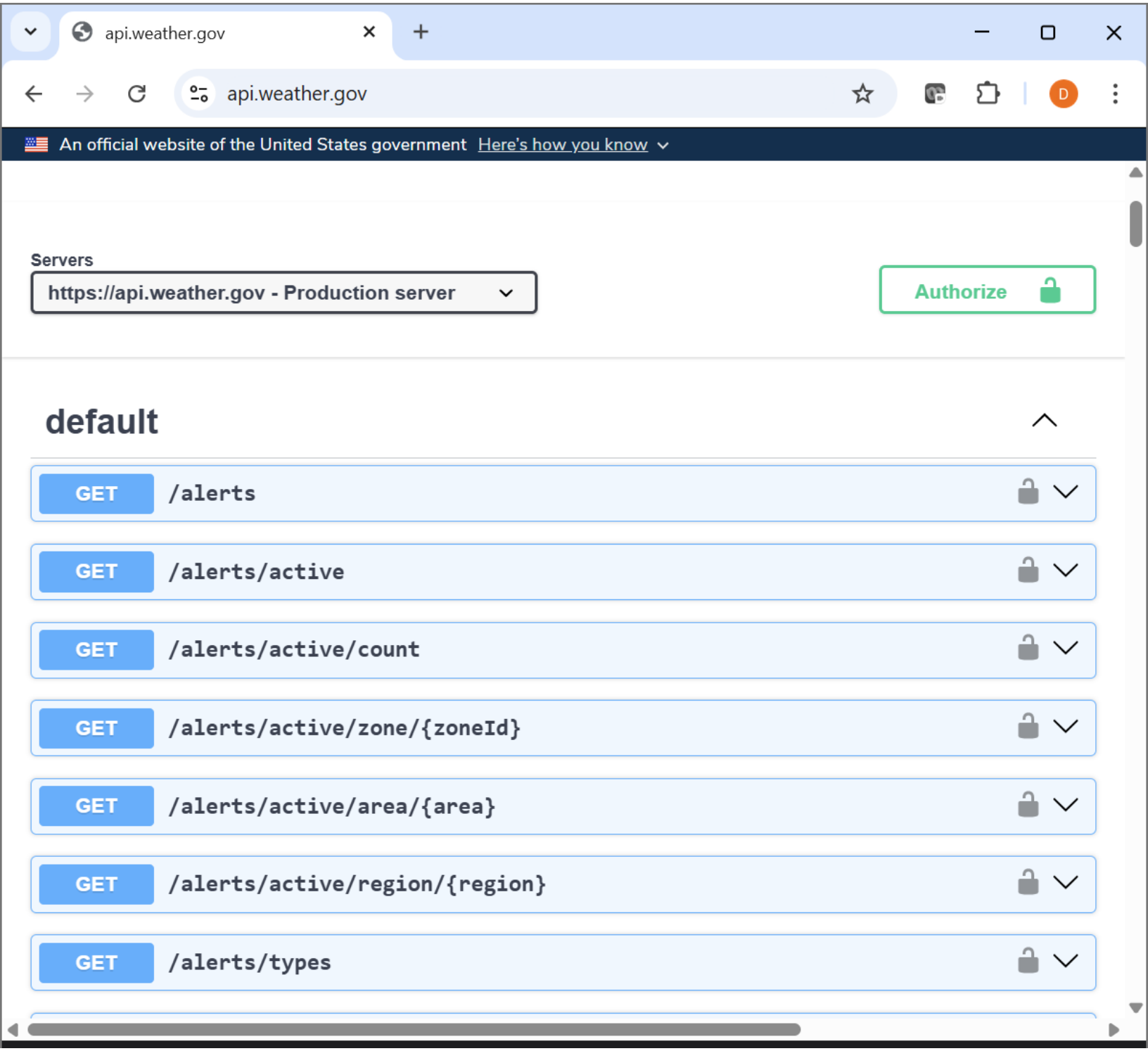
The “sticky dictionary”

In the context of Rhino and Grasshopper scripting, scriptcontext.sticky is a built-in, standard Python dictionary that persists during a Rhino session, allowing you to store and retrieve data between different scripts and even different components.

```python
if sc.sticky.has_key("DRS"):
    print("has key!")
    myInstance = sc.sticky["DRS"]
    print(myInstance)

else:
    print("no key!")
    myInstance = random.random()
    sc.sticky["DRS"] = myInstance
    print(myInstance)
```



Getting data from the Web
example: weather data



Getting data from the Web
getWeather.py

Lots of data is available from the web using “REST APIs”. These often return data in JSON format.

```python
#! python3
#r: requests
import json

import requests

# Define the API endpoint for the National Weather Service
url = 'https://api.weather.gov/gridpoints/MPX/107,71/forecast'

# Send a GET request to the API
response = requests.get(url)

# Check if the request was successful
if response.status_code == 200:
    # Parse the JSON data
    # data = response.json()
    # print(data)
    myText = response.text
    myJson = myText.replace("'", '"')
    myData = json.loads(myJson)
    today = myData.get('properties').get('periods')[0]
    detailed = today.get('detailedForecast')
    print(f"Today's weather is: {detailed}")
else:
    print(f"Failed to retrieve data: {response.status_code}")
```

```
Today's weather is: Mostly sunny, with a high near 57. South southwest wind around 5 mph.
```

Getting data from the Web
getWeather.py

Lots of data is available from the web using “REST APIs”. These often return data in JSON format.


```
{
    "@context": [
        "https://geojson.org/geojson-ld/geojson-context.jsonld",
        {
            "@version": "1.1",
            "wx": "https://api.weather.gov/ontology#",
            "geo": "http://www.opengis.net/ont/geosparql#",
            "unit": "http://codes.wmo.int/common/unit/",
            "@vocab": "https://api.weather.gov/ontology#"
        }
    ],
    "type": "Feature",
    "geometry": {…
    },
    "properties": {
        "units": "us",
        "forecastGenerator": "BaselineForecastGenerator",
        "generatedAt": "2025-03-12T13:07:36+00:00",
        "updateTime": "2025-03-12T10:48:32+00:00",
        "validTimes": "2025-03-12T04:00:00+00:00/P7DT21H",
        "elevation": {
            "unitCode": "wmoUnit:m",
            "value": 269.1384
        },
        "periods": [
            {
                "number": 1,
                "name": "Today",
                "startTime": "2025-03-12T08:00:00-05:00",
                "endTime": "2025-03-12T18:00:00-05:00",
                "isDaytime": True,
                "temperature": 57,
                "temperatureUnit": "F",
                "temperatureTrend": "",
                "probabilityOfPrecipitation": {
                    "unitCode": "wmoUnit:percent",
                    "value": None
                },
                "windSpeed": "5 mph",
                "windDirection": "SSW",
                "icon": "https://api.weather.gov/icons/land/day/sct?size=medium",
                "shortForecast": "Mostly Sunny",
                "detailedForecast": "Mostly sunny, with a high near 57. South southwest wind around 5 mph."
            },
```

Assignment this week

Develop one or more data structures to manage the object data for your project.

Read and write this data to a file, using both

- Structured / tabular data (save as .XLS or CSV)
- Unstructured (JSON) data